How Many Tries Does It Take? Iterative Self-Repair in LLM Code Generation Across Model Scales and Benchmarks

Johin Johny Arimbur
Independent Researcher
johinjohny144@gmail.com

Abstract—Large language models (LLMs) frequently fail to produce correct code on their first attempt, yet most benchmarks evaluate them in a single-shot setting. We investigate iterative self-repair, a strategy in which a model receives its own execution error and is prompted to fix the code, across five open-weight models spanning 8B to 70B parameters and two architectures (dense and mixture-of-experts): Llama 3.1 8B, Llama 3.3 70B, Llama 4 Scout 17B (MoE, 16 experts), Llama 4 Maverick 17B (MoE, 128 experts), and Qwen3 32B. Evaluating on both HumanEval (164 problems) and MBPP Sanitized (257 problems) with up to five attempts per problem, we find that self-repair universally improves pass rates across all models and both benchmarks. On HumanEval, gains range from +4.9 to +14.0 percentage points, while on MBPP gains are larger, ranging from +16.0 to +23.0 percentage points. Notably, Llama 4 Scout (MoE, 16 experts) achieves the highest repair gain on HumanEval (+14.0pp), while Maverick (MoE, 128 experts) and Llama 3.3 70B reach the highest final pass rates on HumanEval (93–94%). Cross-benchmark analysis reveals consistent model rankings and diminishing-returns patterns, with most gains concentrated in the first two repair rounds. Error-type analysis shows that assertion errors (indicating logical mistakes) are the hardest to repair, while syntax and runtime errors are fixed at high rates. A prompt ablation comparing minimal, explain-then-fix, and chain-of-thought repair strategies reveals that richer prompts can further boost repair gains by up to +5.4pp for capable models, though the benefit is model-dependent. The entire experimental protocol runs at zero monetary cost using free-tier API access, demonstrating that iterative self-repair is a practical, training-free technique that substantially improves LLM code generation.

Index Terms—large language models, code generation, self-repair, iterative refinement, HumanEval, MBPP, self-debugging

I. Introduction

The ability of large language models (LLMs) to generate functionally correct code from natural-language specifications has improved dramatically in recent years [1]–[3]. Benchmarks such as HumanEval [1] and MBPP [4] have become standard yardsticks, typically reporting a pass@1 metric: the fraction of problems solved correctly on a single attempt. While informative, this single-shot evaluation protocol diverges sharply from how human programmers actually write code. A professional developer routinely writes an initial draft, runs it, reads the error message, and iterates, often multiple times, before arriving at a correct solution.

Iterative self-repair replicates this workflow within an LLM pipeline. When a generated program fails its test suite, the model receives the error traceback and is asked to produce a corrected version. This cycle can repeat for a fixed number of rounds. Prior work has explored self-repair with proprietary models such as GPT-3.5 and GPT-4 [5], [6], yielding mixed conclusions about its effectiveness. Olausson et al. [5] notably argued that self-repair is "not a silver bullet," observing that weaker models may introduce new errors during repair attempts.

In this paper, we revisit iterative self-repair with five modern open-weight models spanning a range of scales, architectures, and model families: Llama 3.1 8B (8B dense) [7], Llama 3.3 70B (70B dense) [8], Llama 4 Scout 17B (17B active, MoE with 16 experts) [9], Llama 4 Maverick 17B (17B active, MoE with 128 experts) [9], and Qwen3 32B (32B dense) [10]. All experiments are conducted via the Groq free-tier API at zero monetary cost, with greedy decoding (temperature = 0.0) for reproducibility. We evaluate on two benchmarks, HumanEval (164 problems) and MBPP Sanitized (257 problems), with up to five attempts per problem.

Our contributions are as follows:

1) We provide a systematic evaluation of iterative self-repair on five modern open-weight models (2024–2025 generation), demonstrating that self-repair is now universally effective—even at the 8B scale—updating prior findings that weaker models are harmed by self-repair [5].
2) We present a direct comparison of dense and mixture-of-experts architectures for self-repair, revealing that MoE models interact with repair in distinct ways: Scout (16 experts) achieves the highest repair gain (+14.0pp), while Maverick (128 experts) achieves the highest final pass rate (93.9%).
3) We provide cross-benchmark validation on both HumanEval and MBPP Sanitized, showing that model rankings and diminishing-returns patterns are consistent across benchmarks, strengthening the generalizability of our findings beyond single-benchmark studies.

4) We present a fine-grained error-type analysis revealing that error category is a strong predictor of repair success: syntax and runtime errors are repaired at >80% rates, while assertion errors (logical mistakes) succeed at only ∼41%, connecting our findings to the broader literature on the limits of LLM self-correction [11].
5) We quantify the diminishing-returns curve across repair rounds and derive practical deployment guidelines: two repair rounds capture 80–90% of achievable gains, and error-type-based routing can optimize the repair-vs-resample tradeoff.
6) We conduct a repair prompt ablation comparing minimal, explain-then-fix, and chain-of-thought strategies, finding that CoT prompting yields +5.4pp over the minimal baseline, demonstrating that prompt design is a meaningful lever for self-repair.

II. Related Work

A. Code Generation with LLMs

The emergence of Codex [1] demonstrated that LLMs fine-tuned on code repositories could generate functionally correct programs from docstrings. Subsequent work scaled this capability through larger models [2], code-specific pre-training [3], and reinforcement learning from execution feedback [12]. The HumanEval benchmark [1] and the Mostly Basic Programming Problems (MBPP) dataset [4] have become standard evaluation suites, while more challenging benchmarks such as APPS [13] and SPoC [14] test competition-level reasoning.

B. Self-Debugging and Self-Repair

Chen et al. [6] proposed Self-Debug, demonstrating that LLMs can identify and fix bugs in their own code when provided with execution results. Their approach included several feedback strategies: simple error messages, code explanations, and unit test traces. Olausson et al. [5] conducted a thorough investigation of self-repair, concluding that while GPT-4 benefited substantially, weaker models like GPT-3.5 often failed to improve or even degraded in performance. Our work extends this line of inquiry to five modern open-weight models across two benchmarks, revealing that self-repair is now universally effective even at the 8B parameter scale.

C. Iterative Refinement Beyond Code

The principle of iterative self-improvement has been explored in broader contexts. Madaan et al. [15] introduced Self-Refine, a general framework in which LLMs iteratively critique and revise their own outputs across diverse tasks. Shinn et al. [16] proposed Reflexion, using verbal reinforcement signals to improve agent performance over multiple episodes. However, Huang et al. [11] cautioned that LLMs cannot reliably self-correct reasoning without external feedback, an observation consistent with our finding that assertion errors (which require reasoning corrections) are the hardest to repair.

D. Multi-Turn and Collaborative Code Generation

Recent work has also investigated multi-turn interactions for code generation. Key et al. [17] explored multi-turn code generation with instruction-tuned models, while Zhang et al. [18] investigated self-collaboration patterns with ChatGPT. Zhong et al. [19] proposed LDB, a debugger that verifies runtime execution step-by-step. SelfEvolve [20] used LLMs to iteratively evolve code through self-generated feedback. Our work differs in its focus on the simplest possible self-repair protocol (feeding back raw error messages) and its systematic comparison across five models and two benchmarks.

III. Methodology

A. Self-Repair Protocol

Our iterative self-repair protocol operates as follows. Given a programming problem specified by a function signature and docstring (as in HumanEval and MBPP), the model generates an initial code solution at round R_0. This solution is extracted from the model's response, combined with the corresponding test cases, and executed in a sandboxed Python environment. If all test cases pass, the problem is marked as solved. If execution produces an error, we capture the error type and traceback message, then construct a repair prompt containing: (1) the original problem specification, (2) the model's previous code attempt, and (3) the error message. The model is then asked to generate a corrected solution. This cycle repeats for up to four repair rounds (R_1 through R_4), for a maximum of five total attempts per problem.

Formally, for a problem p and model M, let $c_0 = M(p)$ be the initial code generation. If $\text{exec}(c_0)$ succeeds, we record success at R_0. Otherwise, let $e_0 = \text{error}(c_0)$ be the captured error. For each subsequent round $i \in \{1, 2, 3, 4\}$:

$$c_i = M(p, c_{i-1}, e_{i-1}) \tag{1}$$

where the model receives the problem, its previous attempt, and the error message. If $\text{exec}(c_i)$ succeeds at any round, the problem is marked as solved at round R_i and no further attempts are made.

B. Code Extraction

LLM outputs frequently contain natural-language explanations, markdown formatting, and other non-code content surrounding the actual solution. We employ a robust code extraction pipeline that: (1) searches for Python code blocks delimited by markdown fences (‘‘‘python ... ‘‘‘); (2) falls back to extracting the longest contiguous block of syntactically valid Python if no fences are found; and (3) removes any extraneous content such as chain-of-thought traces (e.g., <think>...</think> tags) that some reasoning models may prepend to their outputs.

C. Error Classification

When a solution fails, we classify the error into one of the following categories based on the Python exception type:

- AssertionError: The code runs without crashing but produces incorrect output, failing one or more test assertions. These errors indicate logical mistakes.
- SyntaxError: The generated code is not valid Python.
- TypeError / ValueError: Runtime type mismatches or invalid values, often indicating misunderstanding of the expected input/output types.
- NameError: References to undefined variables or functions, typically caused by incomplete code generation.
- IndexError / KeyError: Out-of-bounds access or missing dictionary keys, indicating edge-case handling failures.
- Timeout: The code does not terminate within the allotted time, suggesting infinite loops or excessive computational complexity.

D. Metrics

Our primary metric is cumulative pass@1 at each round R_i, defined as the fraction of problems solved by round R_i (inclusive of all preceding rounds). Because each problem receives exactly one attempt per round with greedy decoding (temperature = 0.0), pass@1 is deterministic and does not require the unbiased estimator used for stochastic sampling [1]. We also report the self-repair gain, defined as the difference between the final cumulative pass@1 (at R_4) and the initial pass@1 (at R_0), measured in percentage points (pp).

IV. Experimental Setup

A. Models

We evaluate five open-weight models accessed through the Groq free-tier API:

1) Llama 3.1 8B [7]: An 8-billion-parameter dense transformer from Meta. This serves as our smallest baseline and represents the class of efficient, instruction-tuned models commonly deployed in resource-constrained settings.
2) Llama 3.3 70B [8]: A 70-billion-parameter dense transformer from Meta. This is the largest dense model in our evaluation and provides a strong baseline for comparing against mixture-of-experts architectures.
3) Llama 4 Scout 17B [9]: A mixture-of-experts (MoE) model from Meta with 17 billion active parameters distributed across 16 experts. The MoE architecture allows the model to maintain a large total parameter count while activating only a fraction of parameters per token.
4) Llama 4 Maverick 17B [9]: A mixture-of-experts model from Meta with 17 billion active parameters and 128 experts. Compared to Scout, Maverick uses significantly more experts, providing an interesting comparison point for how MoE granularity affects code generation and self-repair.
5) Qwen3 32B [10]: A 32-billion-parameter dense transformer from Alibaba. Qwen3 supports both reasoning (thinking) and non-reasoning modes. Thinking mode was disabled via the /no_think setting to ensure fair comparison with non-reasoning models.

All models are queried with greedy decoding (temperature = 0.0) to ensure deterministic, reproducible outputs. A minimal system prompt is used for all models: "You are an expert Python programmer. Complete the given function. Return ONLY the Python code, no explanations, no markdown formatting." No few-shot examples are provided. For initial generation, each model receives only the function signature and docstring. For repair rounds, the model receives the original problem, its previous code attempt, and the error message. The repair prompt is minimal: it presents the error and asks for a corrected function (see Section III).

B. Benchmarks

We evaluate on two standard code generation benchmarks:

1) HumanEval [1]: 164 hand-written Python programming problems, each with a function signature, docstring, and test cases. Problems range from simple string manipulations to moderately complex algorithmic tasks. All 164 problems were evaluated for all five models.
2) MBPP Sanitized [4]: 257 problems from the Mostly Basic Programming Problems dataset (sanitized subset). MBPP problems tend to be shorter and more focused than HumanEval, but cover a broader range of programming concepts. All 257 problems were evaluated for all five models.

C. Infrastructure

All experiments were conducted using the Groq free-tier API, which provides rate-limited access to hosted open-weight models at no monetary cost. Code execution was performed in isolated Python subprocesses with a 15-second timeout per test case (chosen to accommodate typical HumanEval and MBPP solution runtimes while preventing infinite loops). The total token usage across all HumanEval experiments was approximately 195K tokens for Llama 3.1 8B, 112K for Llama 3.3 70B, 155K for Llama 4 Scout, 93K for Llama 4 Maverick, and 102K for Qwen3 32B.

V. Results

A. HumanEval Results

Table I presents the cumulative pass@1 rates on HumanEval across all repair rounds. Several key observations emerge:

TABLE I
Cumulative Pass@1 (%) on HumanEval (164 problems) by repair round. R_0 is the initial attempt; R_1–R_4 are repair rounds. Δ denotes the total self-repair gain ($R_4 - R_0$) in percentage points.

Model	R_0	R_1	R_2	R_3	R_4	Δ
Llama 3.1 8B	67.1	73.2	75.6	76.2	76.8	+9.8
Llama 3.3 70B	82.9	89.6	90.9	92.7	93.3	+10.4
Scout 17B (16E)	75.6	84.8	87.2	89.0	89.6	+14.0
Maverick 17B (128E)	87.2	91.5	92.7	93.9	93.9	+6.7
Qwen3 32B	87.8	90.2	92.1	92.7	92.7	+4.9

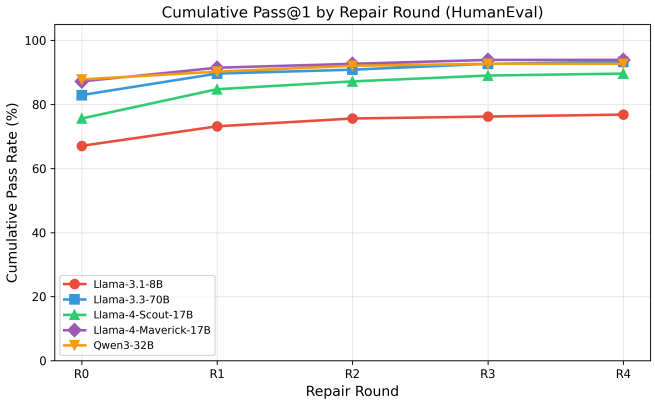

Fig. 1. Cumulative pass@1 rate on HumanEval across repair rounds for all five models. Higher-baseline models show smaller gains, consistent with a diminishing-returns pattern.

Universal improvement. Self-repair improves pass rates for every model tested, confirming its effectiveness across model scales and architectures. Every model gains at least +4.9pp, and the best-performing model (Scout) gains +14.0pp.

Highest final rates. Maverick (93.9%) and Llama 3.3 70B (93.3%) achieve the highest final pass rates, followed closely by Qwen3 32B (92.7%). Despite different architectures and parameter counts, these three models converge to similar final performance levels.

Diminishing returns from higher baselines. Models with higher initial pass rates tend to gain less from repair. Qwen3 32B starts at 87.8% and gains only +4.9pp, while Maverick starts at 87.2% and gains +6.7pp. In contrast, Scout starts lower at 75.6% and gains the most (+14.0pp). This pattern suggests diminishing returns: as the initial pass rate increases, the remaining failures are increasingly composed of hard problems where self-repair is less effective.

Scout's exceptional repair gain. Llama 4 Scout (MoE, 16 experts) achieves the highest repair gain on HumanEval at +14.0pp. This is notable because Scout has fewer experts than Maverick (16 vs. 128) and a lower initial pass rate, yet it benefits most from the repair process. This suggests that Scout's initial errors are more amenable to self-repair than those of higher-baseline models.

Figure 1 visualizes these trajectories. The curves illustrate both the universal upward trend and the convergence of the top three models toward a similar ceiling around 93%.

TABLE II
MBPP Sanitized results: initial pass rate (R_0), final pass rate after repair, self-repair gain (Δ), and number of problems evaluated.

Model	R_0	Final	Δ	Problems
Llama 3.1 8B	55.6	71.6	+16.0	257/257
Llama 3.3 70B	67.7	90.7	+23.0	257/257
Scout 17B (16E)	65.4	83.3	+17.9	257/257
Maverick 17B (128E)	72.0	92.6	+20.6	257/257
Qwen3 32B	70.8	88.3	+17.5	257/257

TABLE III
Cross-benchmark comparison of self-repair gains (Δ, in percentage points) on HumanEval and MBPP Sanitized.

Model	HumanEval Δ	MBPP Δ
Llama 3.1 8B	+9.8	+16.0
Llama 3.3 70B	+10.4	+23.0
Scout 17B (16E)	+14.0	+17.9
Maverick 17B (128E)	+6.7	+20.6
Qwen3 32B	+4.9	+17.5

B. MBPP Results

Table II presents results on MBPP Sanitized. The MBPP results reinforce and extend the findings from HumanEval:

Larger absolute gains. Self-repair gains on MBPP are substantially larger than on HumanEval, ranging from +16.0pp to +23.0pp (compared to +4.9pp to +14.0pp on HumanEval). This is consistent with MBPP having lower initial pass rates, leaving more room for improvement through repair.

Consistent model ranking. The relative ordering of models is consistent across benchmarks. Maverick (92.6%) and Llama 3.3 70B (90.7%) achieve the highest final rates on MBPP, while Llama 3.1 8B has the lowest final rate on both benchmarks. This cross-benchmark consistency strengthens confidence in the findings.

70B achieves largest repair gain. Llama 3.3 70B achieves the largest self-repair gain on MBPP at +23.0pp, improving from 67.7% to 90.7%. This is consistent with the model's strong HumanEval performance (82.9% initial, 93.3% final) and highlights the 70B model's strong repair capability across both benchmarks.

C. Cross-Benchmark Comparison

Table III summarizes the self-repair gains across both benchmarks. MBPP consistently yields higher repair gains than HumanEval for every model. This difference likely reflects two factors: (1) lower initial pass rates on MBPP provide more room for improvement, and (2) MBPP problems, while numerous, tend to be shorter and more focused, making their errors potentially easier to diagnose and repair.

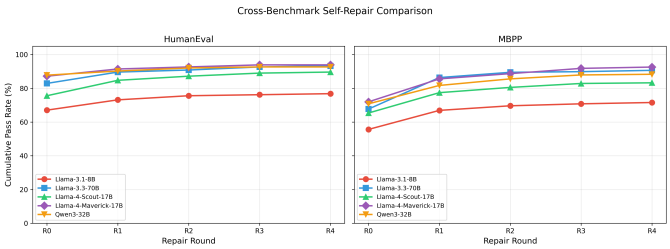

Fig. 2. Cross-benchmark comparison of self-repair gains for all five models. MBPP consistently yields larger gains than HumanEval.

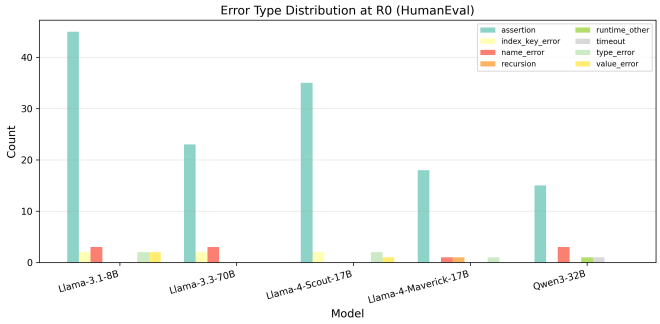

Fig. 3. Distribution of error types at the initial attempt (R_0) on HumanEval for each model.

Figure 2 visualizes the cross-benchmark comparison, highlighting the consistent pattern of larger MBPP gains across all models.

D. Error Distribution Analysis

Figure 3 shows the distribution of error types at the initial attempt (R_0) on HumanEval. The error profiles provide insight into each model's failure modes:

- Llama 3.1 8B: The majority of failures are assertion errors, indicating that the model generates syntactically valid but logically incorrect code. A small number of name, type, and index errors also appear.
- Llama 3.3 70B: Fewer total failures than the 8B model, with assertion errors again dominating. The 70B model's lower error count reflects its stronger baseline capability.
- Scout 17B: A similar profile to the dense models, with assertion errors as the primary failure mode. The MoE architecture does not qualitatively change the error distribution.
- Maverick 17B: The fewest initial failures among all models, again dominated by assertion errors. The higher expert count (128 vs. 16 in Scout) correlates with fewer errors overall.
- Qwen3 32B: With thinking disabled via /no_think, Qwen3 exhibits an error profile similar to the Llama models, with assertion errors as the dominant failure type. When thinking mode is enabled, Qwen3 may exhibit different error profiles due to reasoning-trace contamination of the code output.

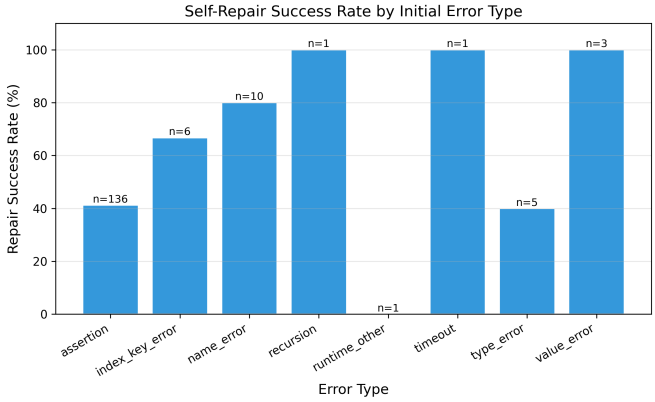

Fig. 4. Repair success rate by error type across all models. Assertion errors (logical mistakes) are the hardest to repair.

E. Repair Success by Error Type

Figure 4 analyzes repair success rates broken down by error type. A clear hierarchy emerges:

- Syntax errors are repaired at near-perfect rates ($> 95\%$). When a model is shown that its output contains invalid syntax, it can almost always produce a corrected version. Syntax errors provide highly informative, actionable feedback.
- Type errors and name errors are repaired at moderate-to-high rates (roughly 60–80%). These errors provide specific diagnostic information (e.g., "NameError: name 'x' is not defined") that guides the model toward the fix.
- Assertion errors are the hardest to repair, with success rates substantially below those of other error types. An assertion error indicates that the code ran successfully but produced the wrong output. The error message provides minimal diagnostic information about what the code computed incorrectly or why, requiring the model to re-examine its logic and identify the conceptual mistake.

This finding is consistent with Huang et al.'s observation [11] that LLMs struggle to self-correct reasoning without external feedback. The pattern holds across all five models and both benchmarks, confirming that error type is a robust predictor of repair success regardless of model scale or architecture.

F. Per-Round Improvement

Figure 5 shows the marginal improvement at each repair round on HumanEval. The pattern of diminishing returns is consistent across all models:

- $R_0 \rightarrow R_1$ (first repair): The largest single-round gain for all five models. Scout gains +9.2pp (the largest first-round gain), while Qwen3 gains +2.4pp (the smallest, reflecting its high baseline).
- $R_1 \rightarrow R_2$ (second repair): Still meaningful but reduced for all models.

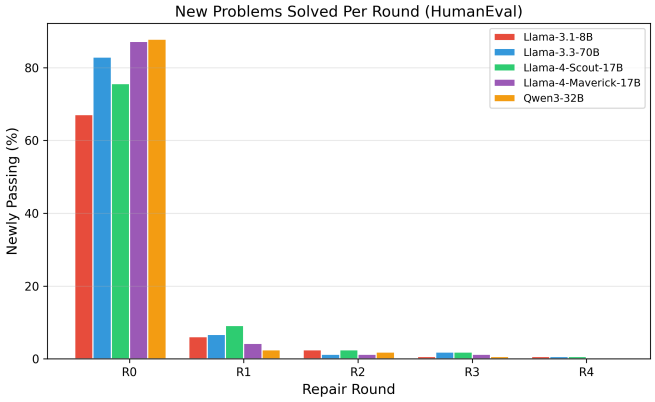

Fig. 5. Marginal improvement in pass@1 per repair round on HumanEval. The first repair round ($R_0 \rightarrow R_1$) yields the largest gain for all models.

TABLE IV
Total token usage and self-repair gain on HumanEval (164 problems).

Model	Total Tokens	Δ (pp)	Tokens/pp
Llama 3.1 8B	195K	+9.8	19.9K
Llama 3.3 70B	112K	+10.4	10.8K
Scout 17B (16E)	155K	+14.0	11.1K
Maverick 17B (128E)	93K	+6.7	13.9K
Qwen3 32B	102K	+4.9	20.8K

- $R_2 \rightarrow R_3$ and $R_3 \rightarrow R_4$ (third and fourth repairs): Minimal additional gains across all models. For Qwen3 and Maverick, R_4 yields no additional improvement over R_3, indicating that these models have solved all problems amenable to self-repair by round 3.

The concentration of gains in early rounds has practical implications: for most applications, two repair rounds capture the bulk of the benefit, and additional rounds offer diminishing marginal returns relative to their computational cost.

G. Token Cost Analysis

Table IV reports total token usage for each model on HumanEval. Llama 3.3 70B achieves the best cost-effectiveness at 10.8K tokens per percentage point of improvement, closely followed by Scout at 11.1K tokens/pp. The 8B model and Qwen3 are less token-efficient, requiring approximately 20K tokens per percentage point. Maverick uses the fewest total tokens (93K) but, due to its smaller gain (+6.7pp), achieves moderate token efficiency (13.9K tokens/pp). Notably, all experiments were conducted at zero monetary cost using the Groq free-tier API.

Table V provides a fine-grained breakdown of when problems are first solved. The first repair round (R_1) accounts for the majority of all repairs across every model. Repair success rate—the fraction of initially-failed problems that are eventually fixed—correlates positively with model capability: Llama 3.3 70B repairs 60.7% of its failures, while Llama 3.1 8B repairs only 29.6%.

TABLE V
Per-round repair breakdown on HumanEval: number of problems first solved at each round, and the repair success rate (fraction of initially-failed problems eventually repaired).

Model	R_0	R_1	R_2	R_3	R_4	Never	Repair %
Llama 3.1 8B	110	10	4	1	1	38	29.6
Llama 3.3 70B	136	11	2	3	1	11	60.7
Scout 17B (16E)	124	15	4	3	1	17	57.5
Maverick (128E)	143	7	2	2	0	10	52.4
Qwen3 32B	144	4	3	1	0	12	40.0

TABLE VI
Repair prompt ablation on HumanEval. Cumulative pass@1 (%) at each round for three prompt strategies on two models.

Model	Strategy	R_0	R_1	R_2	Δ
Llama 3.3 70B	Chain-of-thought	84.1	93.9	96.3	+12.2
	Explain-then-fix	82.9	90.2	93.9	+11.0
	Minimal	82.9	89.6	90.9	+7.9
Scout 17B (16E)	Chain-of-thought	79.3	87.2	89.6	+10.4
	Explain-then-fix	78.0	87.2	87.2	+9.1
	Minimal	75.6	84.8	87.2	+11.6

H. Repair Prompt Ablation

Our main experiments use a minimal repair prompt that provides only the error message and asks for a corrected function. To assess sensitivity to prompt design, we compare three repair prompt strategies on two models—Llama 3.3 70B (dense, 70B) and Llama 4 Scout (MoE, 17B active)—across all 164 HumanEval problems with up to two repair rounds (R_0–R_2):

1) Minimal: The baseline strategy used throughout this paper. The repair prompt presents the error message and requests a corrected function.
2) Explain-then-fix: The model is asked to first explain the bug in 1–2 sentences, then provide the corrected code. This follows the explain-before-fix paradigm from Self-Debug [6].
3) Chain-of-thought (CoT): The model is prompted to reason step-by-step: (1) what does the error tell us, (2) what is the root cause, (3) what is the fix. The corrected code follows the analysis.

Table VI and Figure 6 present the results. For Llama 3.3 70B, the ranking is clear: CoT (96.3%) > explain (93.9%) > minimal (90.9%), with a +5.4pp gap between CoT and minimal after two repair rounds. All three strategies share similar R_0 baselines (82.9–84.1%), confirming that the differences arise from repair effectiveness rather than initial generation quality.

For Scout, the picture is more nuanced. CoT achieves the highest final rate (89.6%), but minimal repair is competitive (87.2%) and actually achieves a higher repair gain (+11.6pp vs. +10.4pp) due to its lower starting point.

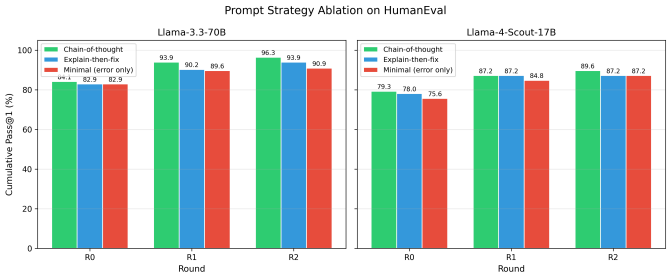

Fig. 6. Prompt strategy ablation on HumanEval for Llama 3.3 70B and Llama 4 Scout. The benefit of richer prompts is model-dependent: substantial for the 70B model, modest for Scout.

The explain-then-fix strategy plateaus at R_1 for Scout, with no additional improvement at R_2 (87.2% at both rounds).

These results reveal an interaction between model capability and prompt strategy: richer prompts provide the largest benefit for stronger models that can leverage the additional reasoning structure. For weaker models, the overhead of generating explanations may offset the benefit, particularly when the model lacks the reasoning capacity to produce useful self-diagnoses. The minimal prompt used in our main experiments thus represents a lower bound on achievable repair gains for capable models, while remaining competitive for smaller models.

VI. Discussion

A. Self-Repair as a Universal Improvement Strategy

Our results demonstrate that iterative self-repair with error feedback is a universally effective strategy for improving LLM code generation across all five models tested, spanning 8B to 70B parameters and both dense and MoE architectures. This finding partially contradicts the conclusions of Olausson et al. [5], who found that weaker models could be harmed by self-repair. The discrepancy likely reflects improvements in instruction-following capabilities of recent model generations: all five models in our study demonstrate competent error comprehension, even at the 8B scale.

The cross-benchmark consistency of our findings strengthens this conclusion. Self-repair improves every model on both HumanEval and MBPP, with consistent model rankings across benchmarks. This suggests that self-repair effectiveness is a robust property of these models rather than an artifact of a particular benchmark.

B. Dense vs. Mixture-of-Experts Architectures

Our evaluation includes both dense models (Llama 8B, 70B, and Qwen3 32B) and MoE models (Scout 16E and Maverick 128E), enabling architectural comparisons. Several observations are noteworthy:

- Scout (16 experts) achieves the highest repair gain on HumanEval (+14.0pp), suggesting that its MoE architecture, while effective at generating mostly-correct code initially, particularly benefits from the opportunity to repair its errors.
- Maverick (128 experts) achieves among the highest final pass rates (93.9% on HumanEval, 92.6% on MBPP) despite having the same active parameter count as Scout (17B). The higher expert count appears to improve both initial code quality and final performance.
- The 70B dense model achieves comparable final performance to Maverick, suggesting that for code generation, a large dense model can match a smaller MoE model with many experts.

These comparisons are suggestive but must be interpreted cautiously: Scout and Maverick differ not only in expert count but also in training data, hyperparameters, and other confounds that we cannot isolate. We report these observations as correlations rather than causal claims about MoE architecture. Controlled experiments varying only the number of experts would be needed to establish a causal relationship between MoE granularity and self-repair effectiveness.

C. Qualitative Analysis of Repair Outcomes

To complement our quantitative findings, we examine representative success and failure cases from HumanEval.

Successful repairs. On HumanEval/38, Llama 3.3 70B produced code that failed at R_0 with a NameError (undefined variable) and repaired it in a single round (R_1). This exemplifies the "easy fix" category: the error message directly pinpoints the problem. More interestingly, on HumanEval/26, Llama 3.1 8B failed with assertion errors for three consecutive rounds (R_0–R_2) before finally producing a correct solution at R_3, demonstrating that persistence can resolve even logical errors. On HumanEval/77, Scout exhibited error-type mutation across rounds (type error $\rightarrow$ assertion $\rightarrow$ type error $\rightarrow$ pass), showing the model genuinely restructuring its approach rather than making superficial edits.

Persistent failures. On HumanEval/10, Llama 3.1 8B produced assertion errors on all five attempts, never resolving the underlying logic error. On HumanEval/32, Maverick exhibited error thrashing: assertion $\rightarrow$ name error $\rightarrow$ assertion $\rightarrow$ assertion $\rightarrow$ timeout, introducing new bugs at each repair attempt—the failure mode described by Olausson et al. [5]. We identified several "universally hard" problems (e.g., HumanEval/132, HumanEval/145) that defeated all five models even with four repair rounds, suggesting these problems require capabilities beyond what current models can access through self-repair alone.

Repair success rates by model. The fraction of initially-failed problems that were eventually repaired varies substantially by model capability: Maverick repaired 52.4% of its failures (11/21), while Llama 3.1 8B repaired only 29.6% (16/54). This confirms that stronger models not

only fail less often but also repair their failures more effectively.

D. Diminishing Returns and the Baseline Effect

A clear pattern in our results is that models with higher initial pass rates gain less from self-repair. On HumanEval, the correlation is striking: Qwen3 (87.8% baseline, +4.9pp gain), Maverick (87.2%, +6.7pp), 70B (82.9%, +10.4pp), Scout (75.6%, +14.0pp), 8B (67.1%, +9.8pp). The one partial exception (8B gaining less than Scout despite a lower baseline) likely reflects the 8B model's weaker error comprehension limiting its repair capability.

This diminishing-returns pattern has a straightforward explanation: as initial pass rates increase, the remaining failures are increasingly composed of genuinely hard problems where the model's errors reflect fundamental reasoning limitations rather than easily-correctable mistakes. These hard problems produce assertion errors, which, as our error analysis shows, are the most resistant to self-repair.

E. MBPP vs. HumanEval: Benchmark Effects

MBPP consistently yields larger self-repair gains than HumanEval across all models. We attribute this to two factors:

1) Lower baselines: Initial pass rates on MBPP are lower than on HumanEval for every model, providing more room for improvement.
2) Problem characteristics: MBPP problems tend to be shorter and more focused, with clear input-output specifications. Errors on these problems may be more diagnosable from error messages, making them more amenable to self-repair.

The cross-benchmark consistency in model rankings, despite different absolute numbers, validates that our findings capture genuine properties of the models rather than benchmark-specific artifacts.

F. Practical Implications

Our findings suggest several practical guidelines for deploying LLM code generation systems:

1) Always include at least one repair round: The first repair round consistently provides the largest marginal improvement for all models tested. The cost of a single additional inference call is negligible compared to the benefit.
2) Two rounds capture most gains: For cost-sensitive applications, two repair rounds (R_0 through R_2) capture approximately 80–90% of the total achievable improvement. Additional rounds beyond this provide minimal marginal benefit.
3) Consider error type for routing: If the initial error is a syntax or type error, repair is very likely to succeed and should always be attempted. If the initial error is an assertion error, the expected improvement is lower, and alternative strategies (e.g., sampling multiple independent solutions) may be more cost-effective.
4) Model selection matters: For applications where final pass rate is paramount, Maverick or a large dense model (70B) should be preferred. For applications where self-repair gain per token is important, Scout and the 70B model offer the best efficiency.

G. Threats to Validity

Internal validity. Our use of greedy decoding (temperature = 0.0) ensures deterministic, reproducible results: rerunning the same experiment produces identical outputs. However, this means our results reflect a single point in the sampling distribution rather than an expected value. We do not report confidence intervals because, under greedy decoding, there is no sampling variance to measure—each problem has exactly one outcome per model. This is a deliberate methodological choice that trades statistical richness for perfect reproducibility. We note that prior work in this area [5], [6] similarly reports single-run results under fixed decoding settings. Our prompt ablation (Section V, Table VI) shows that repair prompt design does affect outcomes: CoT prompting yields +5.4pp over our minimal baseline on Llama 3.3 70B, though the effect is smaller for Scout (+2.4pp). This means the main results reported in this paper represent a lower bound for capable models. The ablation was conducted on two models (one dense, one MoE); prompt sensitivity may vary further across other architectures and scales.

External validity. We evaluate on HumanEval and MBPP, both of which consist of relatively short Python functions. By 2025 standards, these benchmarks are considered "easy," and most strong models achieve high initial pass rates. Self-repair dynamics may differ substantially on harder benchmarks such as APPS [13], LiveCodeBench, or BigCodeBench, where errors more often reflect deep algorithmic misunderstandings rather than surface-level bugs. Our model selection (five models from two families) spans a meaningful range but may not capture behaviors of all architectures. In particular, we do not evaluate proprietary models (GPT-4o, Claude, Gemini), code-specialized models (e.g., DeepSeek-Coder), or models with native tool use. Additionally, because HumanEval and MBPP are widely known benchmarks, some or all of the evaluated models may have been exposed to these problems during pre-training, which could inflate initial pass rates. However, this contamination concern affects all models equally and does not invalidate the relative comparisons between repair strategies.

Construct validity. Our cumulative pass@1 metric counts a problem as solved if it passes all provided test cases. However, test suites for HumanEval and MBPP may not cover all edge cases, meaning some "passing" solutions could contain latent bugs. Additionally, our error classification relies on Python exception types, which may conflate distinct failure modes (e.g., a TypeError

caused by a missing return statement vs. a genuine type mismatch).

Infrastructure. All models were accessed via the Groq free-tier API, which may serve quantized or otherwise optimized model variants. We did not verify that the served weights exactly match the canonical model releases, so pass rates may differ slightly from results obtained with official model weights on local hardware. Additionally, we do not compare self-repair against the obvious alternative strategy of resampling: for the same token budget spent on repair rounds, one could instead draw multiple independent samples at a higher temperature. A rigorous cost-effectiveness comparison between repair and resampling is an important direction for future work.

Qwen3 configuration. Qwen3's thinking mode was disabled via /no_think to ensure fair comparison with non-reasoning models. However, this may disadvantage Qwen3 relative to its full capability, as reasoning mode could improve both initial generation and repair quality. The effect of this choice on Qwen3's results is not characterized.

VII. Conclusion

We have presented a systematic study of iterative self-repair in LLM code generation across five open-weight models spanning 8B to 70B parameters, two architectures (dense and mixture-of-experts), and two standard benchmarks (HumanEval and MBPP Sanitized). Our experiments demonstrate that self-repair is a universally effective, training-free technique: every model improves on every benchmark, with gains ranging from +4.9pp to +14.0pp on HumanEval and +16.0pp to +23.0pp on MBPP.

Cross-benchmark analysis reveals consistent model rankings and repair patterns, validating the generalizability of our findings. We identified several noteworthy results: Scout (MoE, 16 experts) achieves the highest repair gain on HumanEval (+14.0pp), Maverick (MoE, 128 experts) and the 70B dense model reach the highest final pass rates (91–94%), and higher-baseline models consistently show smaller gains (a diminishing-returns effect). Our error analysis confirms that error type is a strong predictor of repair success, with assertion errors being substantially harder to fix than syntax or runtime errors.

The concentration of gains in the first two repair rounds provides actionable guidance for practitioners: two rounds of repair capture the vast majority of achievable improvement, making self-repair a cost-effective addition to any code generation pipeline. Our prompt ablation further shows that richer repair strategies (chain-of-thought, explain-then-fix) can yield additional gains of up to +5.4pp over minimal prompting for capable models, though the benefit is model-dependent and more modest for smaller architectures. Combined with the zero monetary cost of our entire experimental protocol, these results establish iterative self-repair as a practical technique that should be a standard component of LLM code generation systems.

Acknowledgment

The author thanks Groq for providing free-tier API access that enabled the experiments reported in this paper. All experimental code and results are available in the accompanying repository.

References

[1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[2] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago et al., "Competition-level code generation with AlphaCode," in Science, vol. 378, no. 6624, 2022, pp. 1092–1097.

[3] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez et al., "Code Llama: Open foundation models for code," arXiv preprint arXiv:2308.12950, 2024.

[4] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," arXiv preprint arXiv:2108.07732, 2021.

[5] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" arXiv preprint arXiv:2306.09896, 2024.

[6] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," arXiv preprint arXiv:2304.05128, 2024.

[7] A. Grattafiori, A. Dubey, A. Jauhri et al., "Llama 3 model card," arXiv preprint arXiv:2407.21783, 2024.

[8] Meta AI, "Llama 3.3: Large language model," Meta AI Blog, 2024, https://ai.meta.com/blog/llama-3-3/.

[9] ——, "Llama 4: Maverick, scout, and behemoth," Meta AI Blog, 2025, https://ai.meta.com/blog/llama-4-multimodal-intelligence/.

[10] Qwen Team, "Qwen3 technical report," arXiv preprint arXiv:2505.09388, 2025.

[11] J. Huang, C. Xia, P. Shenoy, and J. Zhao, "Large language models cannot self-correct reasoning yet," arXiv preprint arXiv:2310.01798, 2024.

[12] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, "CodeRL: Mastering code generation through pretrained models and deep reinforcement learning," Advances in Neural Information Processing Systems, vol. 35, pp. 21 314–21 328, 2022.

[13] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Zou, D. Song, and J. Steinhardt, "Measuring coding challenge competence with APPS," arXiv preprint arXiv:2105.09938, 2021.

[14] S. Kulal, P. Pasupat, K. Chandra, M. Lee, O. Padon, A. Aiken, and P. Liang, "SPoC: Search-based pseudocode to code," Advances in Neural Information Processing Systems, vol. 32, 2019.

[15] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang et al., "Self-refine: Iterative refinement with self-feedback," Advances in Neural Information Processing Systems, vol. 36, 2023.

[16] N. Shinn, F. Cassano, A. Gopinath, K. Shakkottai, A. Labash, and S. Liu, "Reflexion: Language agents with verbal reinforcement learning," Advances in Neural Information Processing Systems, vol. 36, 2023.

[17] D. Key, S. Lee, J. Shin, and S. Hwang, "Multi-turn code generation with single-turn instruction tuning," arXiv preprint arXiv:2404.11137, 2024.

[18] Y. Zhang, J. Chen, D. Li, and Y. Zheng, "Self-collaboration code generation via ChatGPT," arXiv preprint arXiv:2304.07590, 2023.

[19] L. Zhong, Z. Wang, and J. Shang, "LDB: A large language model debugger via verifying runtime execution step-by-step," arXiv preprint arXiv:2402.16906, 2024.

[20] S. Jiang, Y. Wang, and Y. Wang, "Selfevolve: A code evolution framework via large language models," arXiv preprint arXiv:2306.02907, 2023.